

O Plano de Execução (v2 Otimizado)

Fase 0: A Fundação (O "Setup" do Projeto)

Objetivo: Preparar as nossas ferramentas e contas. Nenhum código funcional será escrito, mas isto é mandatório. **Blueprint Mestre:** Referencia a Seção 2.6 (Ambientes) e Seção 8 (CI/CD).

- 1. Contas AWS: Criar as duas Contas AWS (uma para dev, uma para prod).
 - 2. Repositório GitHub: Criar o repositório ([ia-decifrada](#)).
 - 3. Branches Git: Criar as duas branches de trabalho: [main](#) (protegida) e [develop](#).
 - 4. Autenticação (OIDC): Configurar manualmente o "Provedor OIDC" do GitHub Actions em ambas as contas AWS .
-

Fase 1: O "Esqueleto" (O Primeiro Post Visível)

Objetivo: Provar que a nossa arquitetura core funciona. Queremos ver um único post do blog a ser renderizado num URL público. **Blueprint Mestre:** Referencia o endpoint [GET /post/{slug}](#) e o template [Página de Post Padrão](#).

- **Bloco 1: Infraestrutura de Backend (Core) (Otimizado)**
 - Pedir à IA para gerar o Terraform para criar a **Tabela Posts** (Seção 3.1) e a **Tabela Autores** (Seção 3.3) (apenas no ambiente dev) .
 - Pedir o Terraform para o **API Gateway** (Seção 2.3) para expor os endpoints públicos.
 - (Adição Lógica): Pedir o Terraform para os *recursos* da Lambda (IAM Roles, definições de função) que o API Gateway irá acionar.
- **Bloco 2: Lógica do Backend (Endpoints Públicos)**
 - Pedir o código Lambda (Node.js/TS) para apenas dois endpoints da API Pública (Seção 6.1):
 - [GET /post/{slug}](#)
 - [GET /autor/{id}](#)
- **Bloco 3: Infraestrutura de Frontend (Core)**
 - Pedir o Terraform para a infra básica do Frontend (Seção 2.2) (S3, CloudFront, Lambda@Edge via OpenNext) .
- **Bloco 4: Frontend (Fundação e Layout) (Otimizado)**
 - (Adição Lógica): Pedir o componente [Layout.js](#) principal do Next.js, que irá importar o CSS Global (Seção 5.1) e as Fontes (Seção 5.2).
 - Pedir à IA para gerar os componentes [Header/Navbar](#) (5.10) e [Footer](#) (a ser definido).

- **Bloco 5: Frontend (Página de Post)**
 - Pedir à IA para gerar os componentes **AuthorBox** (5.7) e **TOCBox** (5.8).
 - Pedir à IA para gerar a **Página de Post Padrão** (v1.9), que será "envolta" pelo **Layout** e consumirá os dados da API.
 - **Bloco 6: "Seed" Manual (O Seu Trabalho)**
 - Você (Marcelo) acede à consola da AWS (em dev) e adiciona manualmente um item à Tabela Autores e um post de teste à Tabela Posts.
 - **Bloco 7: Deploy (CI/CD - Dev)**
 - Pedir à IA para gerar o workflow do GitHub Actions (Seção 8) que faz o deploy automático da branch **develop** para o ambiente **dev**.
 - **Resultado Visualizável (Fase 1):** Ao fazer push para a branch **develop**, o site <https-dev.iadecifrada.com> será criado. Aceder a </post/o-slug-que-voce-criou-manualmente> irá mostrar o seu primeiro post
-

Fase 2: O "Cérebro" (O CMS Admin Funcional)

Objetivo: Parar de adicionar dados manualmente. Construir a ferramenta de escrita (o Admin). **Blueprint Mestre:** Foco total nas Seções 6.2 (API Admin) e 7 (CMS Admin).

- **Bloco 8: Infraestrutura do Admin (Auth)**
 - Pedir o Terraform para o **Amazon Cognito** (Seção 2.3).
 - Pedir o Terraform para a infra do **Admin UI** (S3 privado + CloudFront restrito) (Seção 7).
- **Bloco 9: Feature de Mídia (API + Processamento) (Otimizado)**
 - **Backend:** Pedir o código Lambda para o endpoint **POST** </admin/media/upload-url>.
 - **Infra + Backend:** Pedir o Terraform para a **ImageProcessorLambda** (com o trigger S3) e o código-fonte (Node.js + Sharp.js) para ela.
- **Bloco 10: Backend (Admin CRUD)**
 - Pedir à IA para gerar o código Lambda para os endpoints restantes da API Admin (Seção 6.2):
 - CRUD de Posts (incluindo a lógica de status "Programado").
 - CRUD de Categorias.
 - CRUD (Upsert) de Autores.
- **Bloco 11: Infra (API Admin + Agendamento)**
 - Pedir ao Terraform para atualizar o API Gateway, expondo todos estes novos endpoints /admin/* e protegendo-os com o Autorizador do Cognito.
 - Pedir o Terraform para a **PostSchedulerLambda** (com o EventBridge) e o seu código-fonte.
- **Bloco 12: Frontend (Admin UI)**
 - Pedir à IA para gerar o Protótipo Funcional do CMS Admin (como um app **Vue.js** estático), conectando-o aos endpoints da API Admin.

- **Resultado Visualizável (Fase 2):** A página <https-admin-dev.iadecifrada.com> está funcional. Você pode fazer login, escrever um novo post, agendá-lo, fazer upload de uma imagem e editar o seu perfil .
-

Fase 3: O "Hub" (O Site Público Completo)

Objetivo: Com o CMS funcional, vamos construir o resto do site público. **Blueprint Mestre:** Foco total nos templates restantes (Seções 4 e 5).

- **Bloco 13: Backend (Endpoints Públicos Restantes)**
 - Pedir à IA para gerar o código para todos os endpoints públicos restantes (Seção 6.1): [/posts/recentes](#), [/posts/populares](#), [/artigos](#), [/categoria/{slug}](#), [/projeto](#), [/busca](#), [/categorias](#) .
 - **Bloco 14: Infra (Atualização API Pública)**
 - Pedir ao Terraform para atualizar o API Gateway e expor todos estes novos endpoints públicos.
 - **Bloco 15: Frontend (Site Público Completo)**
 - Pedir à IA para gerar os Componentes de UI restantes (Seção 5): [PostCard](#), [Pagination](#), [AdSenseBlock](#).
 - Pedir à IA para gerar todos os 8 templates de página restantes (Seção 4.1), conectando-os às suas APIs: [Homepage](#), [Página de Artigos](#), [Página de Categoria](#), [Página "O Projeto"](#), [Página "Sobre"](#), [Página "Serviços"](#), [Página "Newsletter"](#), [Página 404](#), [Página de Busca](#) .
 - **Resultado Visualizável (Fase 3):** O site <https-dev.iadecifrada.com> está 100% completo. A homepage mostra os posts que você escreveu no CMS e todos os links estão funcionais .
-

Fase 4: Produção (O "Go-Live")

Objetivo: Mover o nosso site [dev](#) (completo e testado) para o público. **Blueprint Mestre:** Foco total na Seção 8 (CI/CD) e 2.6 (Ambientes).

- **Bloco 16: DevOps (Pipeline de Produção)**
 - Pedir à IA para finalizar o workflow de CI/CD (Seção 8).
 - O workflow deve agora incluir o trigger para a branch [main](#) (via merge).
 - O job de [prod](#) deve usar os ficheiros [env/prd.tfvars](#), usar a autenticação OIDC de produção e pausar para **Aprovação Manual** antes de aplicar o Terraform.
- **Bloco 17: Lançamento (O Seu Trabalho)**
 - Você faz o merge da branch [develop](#) (estável) para a branch [main](#).

- O pipeline do GitHub Actions começa, roda o `terraform plan` de `prod` e pausa.
- Você revê o plano e clica em "Aprovar".
- O pipeline executa o `terraform apply` na Conta AWS de Produção, criando toda a infra.
- O pipeline faz o deploy do código (Next.js, Lambdas) para `prod`.
- **Bloco 18: Configuração Final**
 - Apontar o seu DNS público para o CloudFront de `prod`.
- **Resultado Visualizável (Fase 4):** O seu site está no ar para o mundo.